

本小节内容:

混合运算
printf 讲解

1 混合运算

类型强制转换场景

整型数进行除法运算时，如果运算结果为小数，那么存储浮点数时一定要进行强制类型转换，请看下面例子

```
#include <stdio.h>
```

```
//强制类型转换
```

```
int main()
```

```
{
```

```
    int i=5;
```

```
    float j=i/2;
```

```
    float k=(float)i/2;
```

```
    printf("%f\n",j);
```

```
    printf("%f\n",k);
```

```
    return 0;
```

```
}
```

j 得到的值为 2, k 得到的值是 2.5，因为当我们整数做除法时，默认进行整除，要得到小数，需要首先进行强制类型转换操作。

2 printf 函数介绍

printf 函数可以输出各种类型的数据，包括整型、浮点型、字符型、字符串型等，实际原理是 printf 函数将这些类型的数据格式化为字符串后，放入标准输出缓冲区，然后将结果显示到屏幕上。

语法如下：

```
#include <stdio.h>
```

```
int printf(const char *format, ...);
```

printf 函数根据 format 给出的格式打印输出到 stdout（标准输出）和其他参数中。

字符串格式（format）由两部分组成：显示到屏幕上的字符和定义 printf 函数显示的其他参数。我们可以指定一个包含文本在内的 format 字符串，也可以是映射到 printf 的其他参数的“特殊”字符，如下列代码所示：

```
int age = 21;
```

```
printf("Hello %s, you are %d years old\n", "Bob", age);
```

代码的输出如下：

```
Hello Bob, you are 21 years old
```

其中，%s 表示在该位置插入首个参数（一个字符串），%d 表示第二个参数（一个整数）应该放在哪里。不同的 %codes 表示不同的变量类型，也可以限制变量的长度。printf 函数的具体代码格式如下表所示。

printf 函数的具体代码格式

代 码	格 式
%c	字符
%d	带符号整数
%f	浮点数
%s	一串字符
%u	无符号整数
%x	无符号十六进制数，用小写字母
%X	无符号十六进制数，用大写字母
%p	一个指针
%%	一个 '%' 符号

位于 % 和格式化命令之间的一个整数被称为最小字段宽度说明符，通常会加上空格来控制格式。

- 用 %f 精度修饰符指定想要的小数位数。例如，%5.2f 会至少显示 5 位数字并带有 2 位小数的浮点数。
- 用 %s 精度修饰符简单地表示一个最大的长度，以补充句点前的最小字段长度。

printf 函数的所有输出都是右对齐的，除非在 % 符号后放置了负号。例如，%-5.2f 会显示 5 位数字、2 位小数位的浮点数并且左对齐。

下面来看一个例子，如下面例子所示。

【例】printf 函数输出对齐。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i=10;
```

```
    float f=96.3;
```

```
    printf("student number=%3d score=%5.2f\n",i,f);
```

```
    printf("student number=%-3d score=%5.2f\n",i,f);
```

```
    printf("%10s\n","hello");
```

```
}
```

执行结果如下图所示, 可以看到整型数 10 在不加负号时靠右对齐, 加负号时靠左对齐, %10s 代表字符串共占用 10 个字符的位置。因为 printf 函数默认靠右对齐, 所以 "hello" 字符串相对于左边的起始位置有 5 个空格的距离。掌握这些内容后, 在做 OJ 作业时, 就会很容易掌握打印格式的控制。

```
student number= 10 score=96.30
```

```
student number=10 score=96.30
```

```
hello
```

【图】 执行结果图